

Scalable Systems 1



Today's Agenda

Difference between DS/Algo and Scalable Systems

Problem solving framework recap

Online processing: Design Instagram clone

Online processing: Design Uber clone



INTERVIEW
KICKSTART

Difference between DSA and System Design



INTERVIEW
KICKSTART

What changes when we move from DS/Algos to SysD?

In DSA, for a given computational problem

- Design a correct & efficient algorithm
- Code it up

Assumptions:

- Only one user
- Program will run only once
- Unbounded CPU and RAM to solve the problem

What changes when we move from DS/Algos to SysD?

In system design,

- Your DS/Algos program is now a service in System Design.
- The users of the service known as **clients**
 - May want to execute program remotely over a network.
 - Will need to *request* your **server** machine to run the program using the inputs they send over the network.
- Your server machine
 - Needs to send the output to of a request as a *response* over the network.
 - Needs to always be actively listening for requests.
- Your program needs to be **scalable** if it can handle a huge growth in the load (number of concurrent requests) without sacrificing the *response time*.

The standalone program is converted into an **online service** and we will see how to build such an online service.



Scalability and its types

Scalability is property of a system to handle growing amount of traffic.

The scaled program with client-server will act as an online service

In case the program is scaled offline, it is known as batch processing

When the input is not fixed and comes as a stream, it is known as stream processing

Top-Down steps for System Design in an interview



Step 1: Gather all requirements of the system

1. Gather functional requirements.
 - a. This is the detailed problem statement.
 - b. Describe user's view of the system in plain English
 - c. Shows that you can communicate and unpack an open-ended problem.
 - d. Ask clarifying questions.
2. Design Constraints / Scalability requirements
 - a. Number of users, transactions per second etc
 - b. The interviewer may give them or may ask the candidate.
 - c. Can be collected at a later step as well.

Step 2: Defining Microservices of the System

A Microservice

- Is an *independently deployable* service modeled around a business domain.
- *Loosely coupled*, you can change one microservice and deploy it into production without affecting other microservices.
- Each microservice is a *black box* to other microservices.
- Small and focused on *doing one thing well*: Owned by a single (2 pizza) team
- Communicate with each other using language-independent network APIs (e.g. REST with HTTP)

Step 2: Defining Microservices of the System

Bucketize the functional requirements into microservices.

1. Cluster the requirements such that each cluster can be handled by a different team.
2. Rule of thumb: If the APIs and data models for two functional requirements do not look the same, put them in different buckets.
3. This is subjective, depends on the individual
4. After doing this, you will know whether it is a depth-oriented problem (~1 microservice) or breadth-oriented problem (many microservices)

Step 3: Design the Logical Architecture

1. The logical architecture is a block diagram with one block for each microservice
2. Draw and explain the data/logic flow between them
3. Rules of thumb:
 - a. If the client (user or microservice) is waiting for response from the microservice, use HTTP/REST APIs
 - b. If the client microservice does not expect an immediate response from the server microservice, use a message queue (pub-sub) which is its own microservice.
 - c. If data transfer is offline, you may use batched ETL (Extract Transform Load) jobs

Step 4: Deep dive into each microservice

High-level microservice architecture:

- In general each microservice can have, upto three tiers:
 - Backend tier with business logic (App server) -- this is where your single-machine code conceptually goes
 - Cache tier for faster response time (Cache) - data here is a subset of the database tier, but can be stored in any format/data structure
 - Storage tier for data storage and retrieval (Database): store in row-major/col-major/files (for movies and other unstructured formats)

Note: We don't worry about front-end tier



Step 4: Deep dive into each microservice

1. If there are many microservices, discuss with the interviewer which ones to focus on.
2. Budget your time per microservice, since you have only a constant amount of time
3. If there is only 1 microservice, there might be a coding part to the interview.
4. For each microservice
 - a. Solve each tier logically (as if solving for a single server)
 - b. Identify Data Model (what data needs to be stored) to match the functional requirements
 - c. Discuss how data will be stored in storage and cache tiers
 - d. Propose APIs to match the functional requirements
 - e. Propose workflow/algo for the the APIs in each tier

Note: Scale is not in the picture yet.



Step 4: Deep dive into each microservice

- Propose flow across tiers within the microservice
 - Most of technical design interviews have meat in this phase
 - Non-deterministic, every candidate will come with his or her own proposal
 - Changes from problem to problem
 - This constitutes the 'most thinking' portion of the interview
 - Most chances to flunk here unless candidate is prepared

Step 5: Identify the need for scale

In each microservice, identify the need for scale

- A deterministic set of reasons can be posed across all interviews
 - Need to scale for storage (storage and cache tiers)
 - Need to scale for throughput (CPU/IO)
 - Need to scale for API parallelization
 - Need to remove hotspots
 - Availability and Geo-distribution
- The constraints or numbers change from problem to problem
- Solve algebraically first and then put numbers. Algebraic formula is same for all problems
- If phase 1 takes more time, don't spend time on calculations/estimations here unless asked for



Scaling Options: Vertical scaling

- Easiest way to scale to higher load
- One powerful machine (with lots of CPUs, RAM chips and disks), at a single location
- Cost eventually grows faster than linearly.
- Has a ceiling

Performance of one machine depends on:

- Number of CPU cores, speed of each CPU core, competing processes on the same CPU, amount of RAM, type of disk (SSD vs Magnetic Disk), number of disks (parallel access), network connectivity, GPUs.

Scaling up easy with cloud computing (up to 96 cores)



**INTERVIEW
KICKSTART**

Scaling Options: Horizontal scaling

Add more machines. But need to change your program and coordinate tasks on multiple machines across the network.

If the “web application” threads are stateless, can push the state into a database.

This allows us to add more servers and a proxy (load balancer and/or cache).

A few shortcomings:

1. Complex distributed systems (managing consistency, availability, debugging)
2. If not architected properly, can result in fragmented servers, improper division of workload



**INTERVIEW
KICKSTART**

Step 6: Propose the distributed system

Scale the tier and propose the distributed system

- This is the most deterministic portion of the solution
- Steps
 - Draw a generic distributed architecture per tier
 - If app server tier and stateless, just round robin requests
 - If cache or storage tier
 - Partition the data into *shards* or buckets to suit requirements of scale (changes from problem to problem)
 - Propose algorithm to place shards in servers (*consistent hashing*) (same across all problems)
 - Explain how APIs work in this sharded setting (problem specific)
 - Propose *replication* (same across all problems)
 - Propose CP or AP (algorithms for CAP theorem do not change from problem to problem)
- This is how each microservice looks within a single data center, either on-premise or VPC in cloud

Interview Process and Timing

<i>Stage</i>	<i>Stage Length</i>	<i>Time into the discussion</i>
Introduction	~1-5 minutes	~0 minutes
Problem statement	~1-2 minutes	~5 minutes
Functional Requirements & Scalability	~5 minutes	~10 minutes
Microservices	~5 minutes	~15 minutes
Logical Architecture	~5 minutes	~20 minutes
Microservice(s) deep dive	~5-25 minutes	~25 minutes
Recap	1 minute	~45 minutes
Q&A	1-10 minutes	~50 minutes



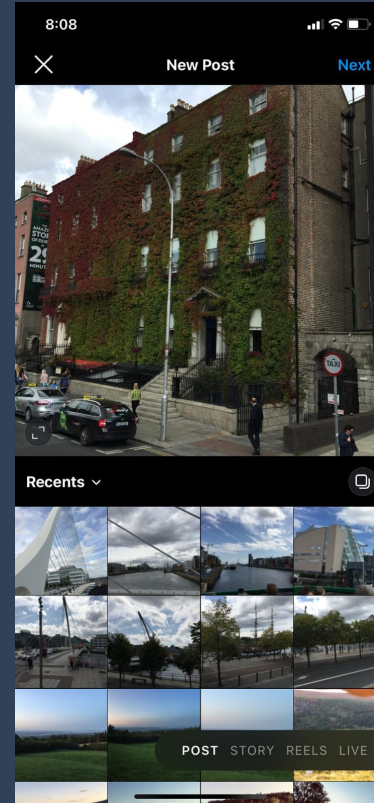
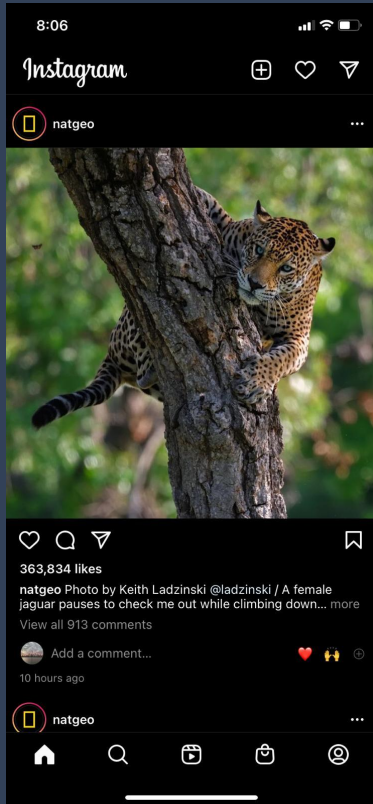
Problem 1

Online processing: Design Instagram clone

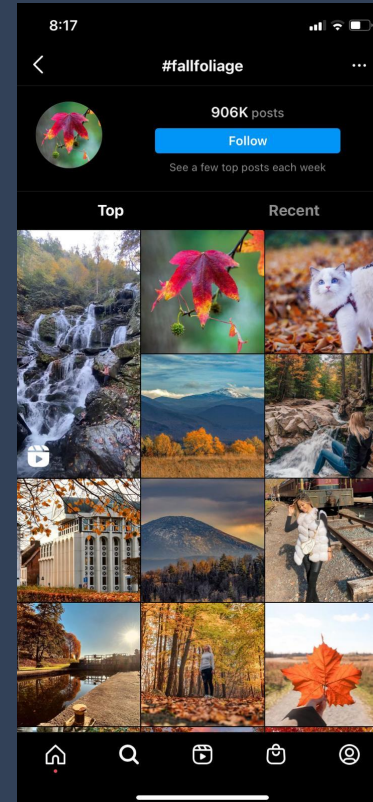
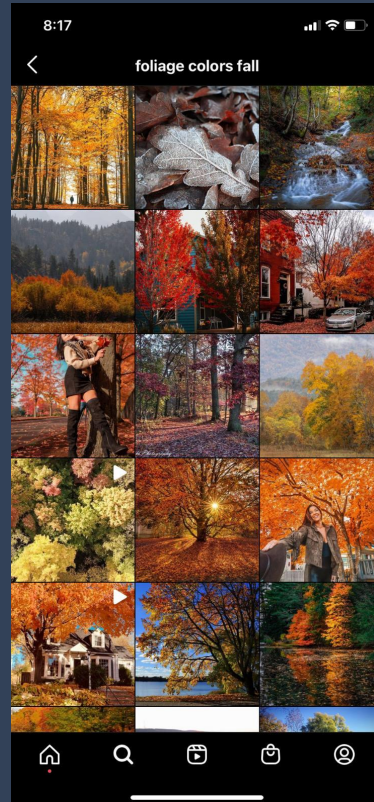
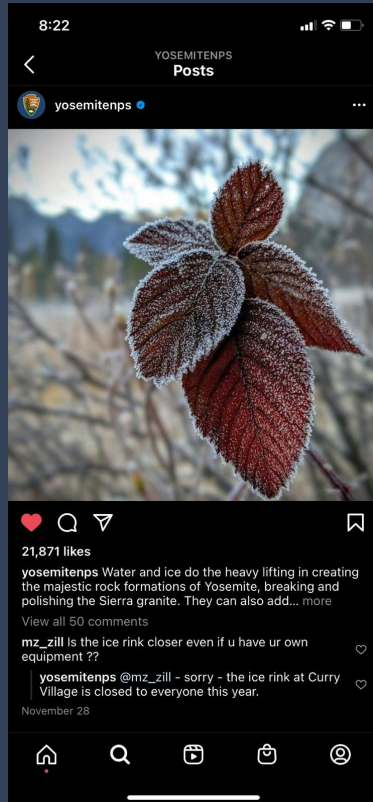


INTERVIEW
KICKSTART

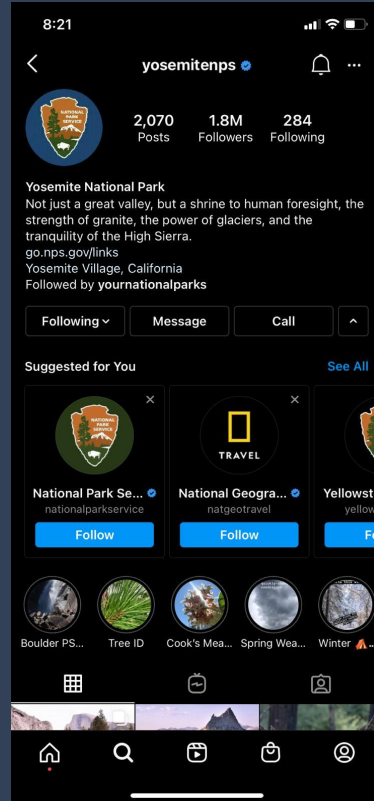
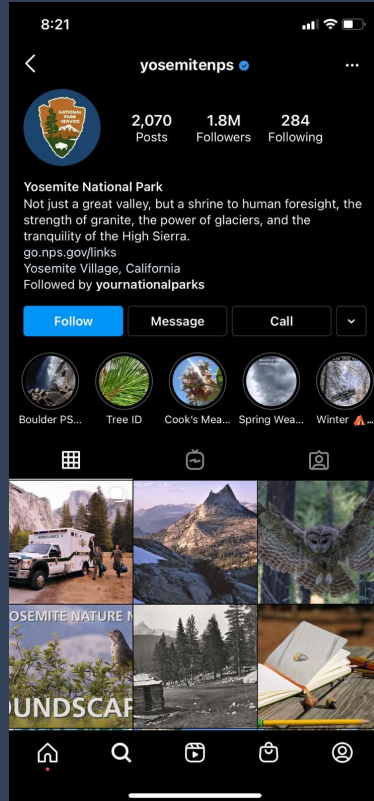
Instagram Application



Instagram Application



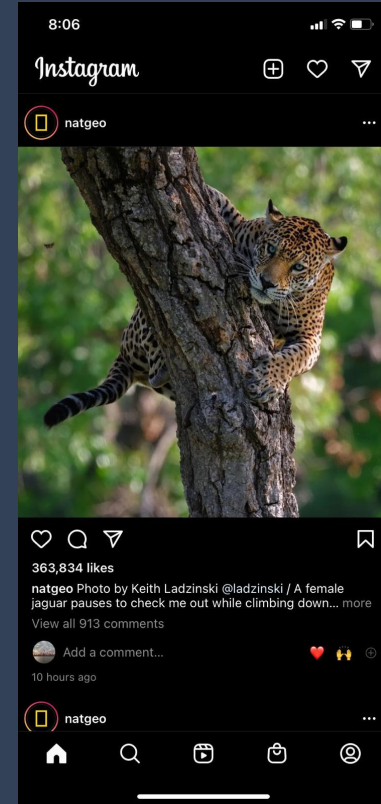
Instagram Application



Step 1: Gathering Requirements

Functional Requirements:

1. View, upload photos.
2. Like, comment on photos.
3. Search for photos based on tags or titles.
4. Follow/unfollow other users.
5. Generate a news feed, based on who the user follows.



Step 1: Gathering Requirements

Non Functional Requirements

- Service needs to be highly available (prefer availability over consistency).
 - Acceptable latency of X milliseconds to generate the news feed, Y milliseconds to view a photo.
 - The systems needs to be highly reliable (no lost photos)
-
1. Total users ~= 1 billion
 2. Daily active users ~= 500 million
 3. New photos per second ~= 1000
 4. Photo views per second ~= 20000
 5. Average photo file size ~= 200kb

Identify APIs according to Functional Requirements

1. Users should be able to view, upload and download photos.
 - a. `viewPhoto(photo_id, user_id)`
 - b. `viewPhotos(user_id)`
 - c. `postPhoto(user_id, photo, location, time_stamp, title, tags[])`
2. Users should be able to like, comment on photos.
 - a. `likePhoto(photo_id, user_id)`
 - b. `postComment(photo_id, user_id, time_stamp, comment)`
3. Users should be able to follow/unfollow other users.
 - a. `follow(follower_id, followee_id, time_stamp)`
 - b. `unfollow(follower_id, followee_id, time_stamp)`

APIs according to Functional Requirements

1. System should generate a news feed for the user, based on who the user follows.
 - a. `generateFeed(user_id, time_stamp, batch_size, batch_range[])`
2. Users should be able to search for accounts or photos based on tags or keywords.
 - a. `searchAccounts(accName, user_id)`
 - b. `searchPhoto(user_id, tags[]/keyword)`
3. User Authentication, account creation. (out of scope)
4. Live/Stories (out of scope)

How to organize the information in memory?

1. Fetch a photo given a photo id
2. Fetch User's information
3. Likes, Comments
4. Follow/Unfollow a user

What key do you need?

How do you translate that to a DB Model?

Data Model according to Functional Requirements

1. Users should be able to view, upload and download photos.
 - a. PhotoTable: Primary Key = Photoid(int64).
Other columns = UserId(int64), description(varchar 64), ImgStorePath(varchar 64), ThumbnailPath(varchar 64), PhotoLatitude(int), PhotoLongitude(int), TimeStamp(datetime)
 - b. UserTable: Primary Key = userId(int64)
Other columns = name(varchar 32), email(varchar 32), dob(datetime), zipcode(int), createTime(datetime), lastLogin(datetime)
2. Users should be able to like, comment on photos.
 - a. LikesTable columns: UserId(int64), Photoid(int64), TimeStamp(datetime)
Primary Key = UserId + Photoid
 - b. CommentsTable columns: Photoid(int64), UserId(int64), Comment(varchar 128)
TimeStamp(datetime)
Primary Key = Photoid + UserId + TimeStamp

Data Model according to Functional Requirements

1. Users should be able to search for photos based on tags or accounts based on name.
 - a. Search on account can use UserTable, indexed on name.
 - b. Search on tags: Reverse index on tags->photo id (part of offline system)
2. Users should be able to follow/unfollow other users.
 - a. FollowTable columns: FollowerId(int64), FolloweeId(int64), TimeStamp(datetime)
Primary Key = FollowerId + FolloweeId
3. News Feed generation:
 - a. UserNewsFeed table: Primary Key = FeedItemId(int64).
Other columns = UserId(int64), PhotoId(int64), PhotoOwnerId(int64), Description(varchar 64), ImgStoreOrCDNPath(varchar 256), FeedCreationTimestamp(datetime), NumLikes(int32), NumComments(int32))

Step 2: Defining Microservices

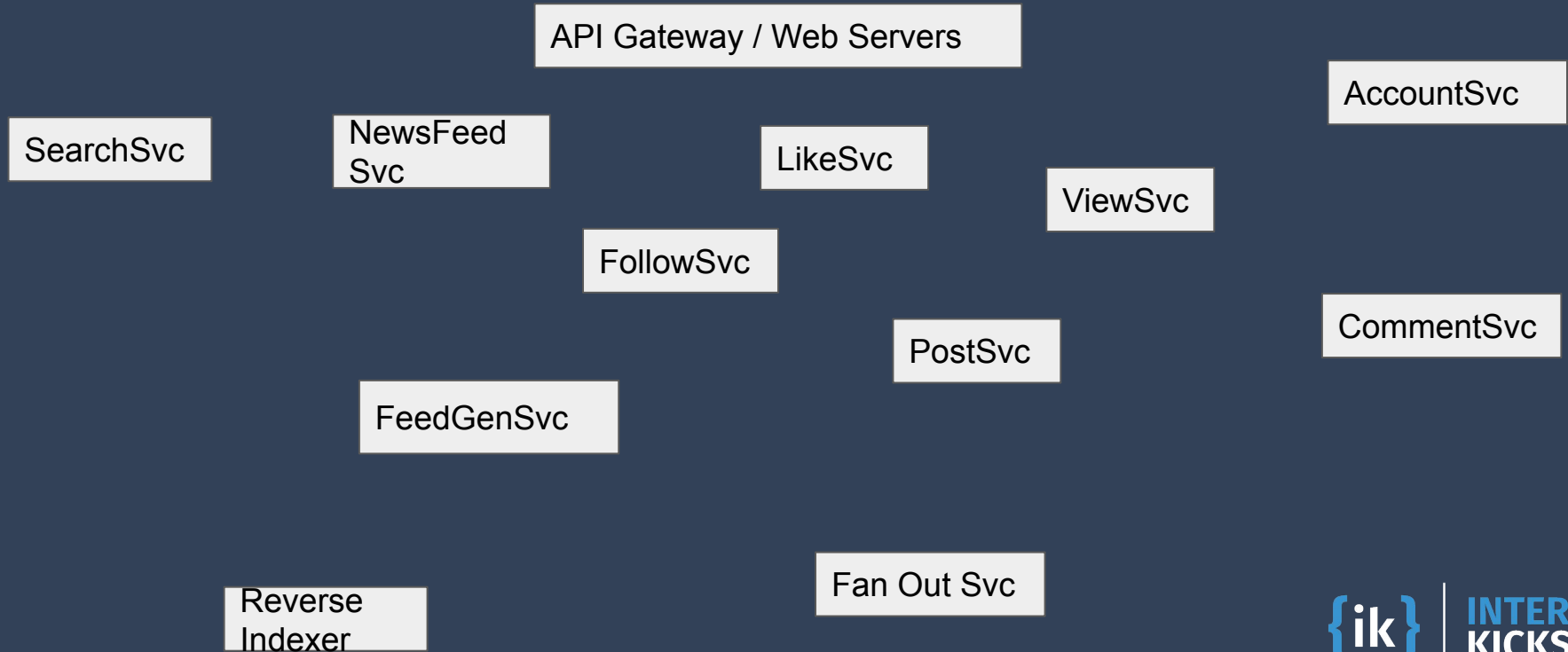
- ViewService
- PostService
- LikeService
- FollowService
- CommentService
- NewsFeedService
- *FeedGenerationService*
- SearchService
- AccountService

Breadth Oriented Problem

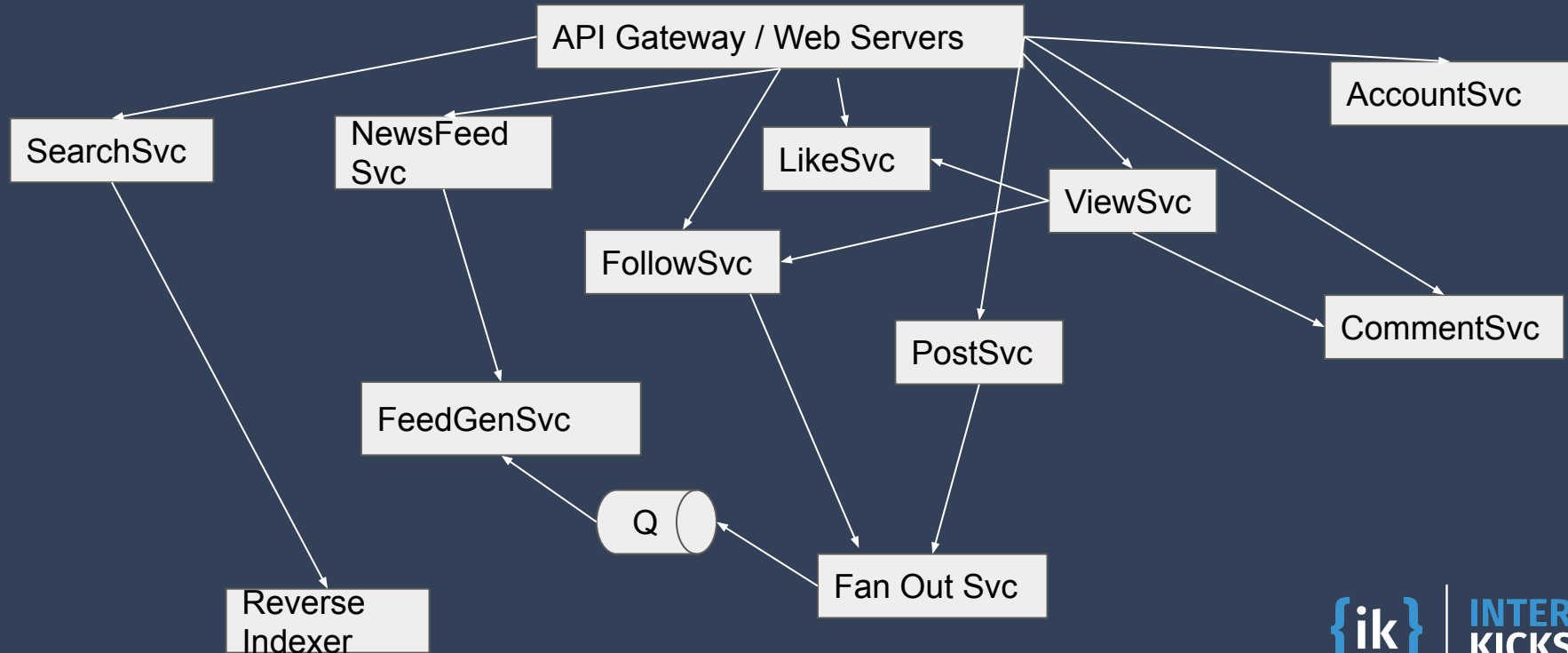


INTERVIEW
KICKSTART

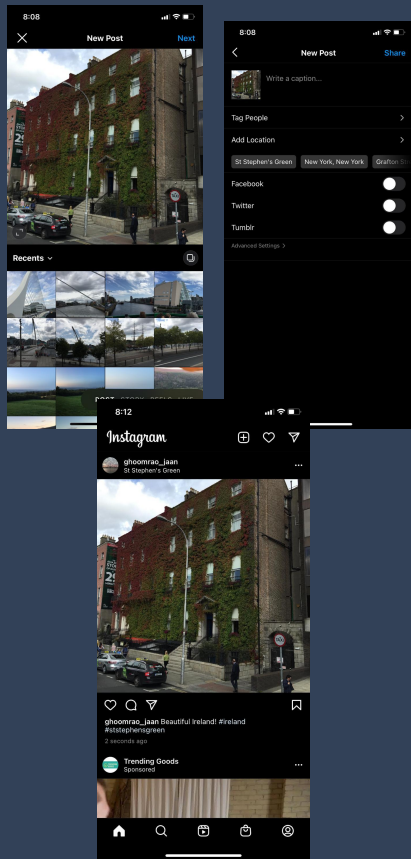
Step 3: Logical block diagram



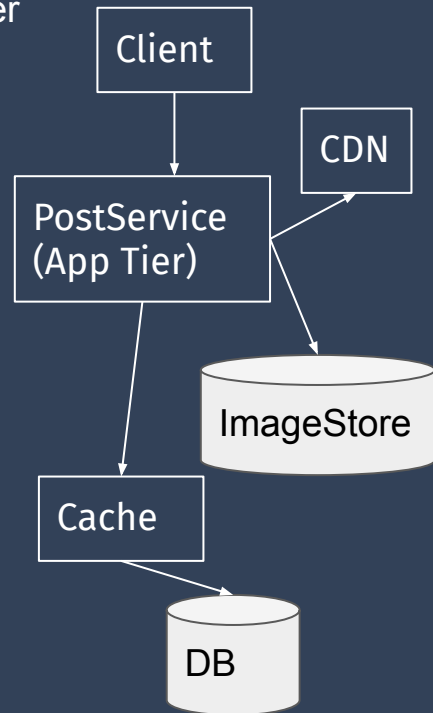
Step 3: Logical block diagram



Step 4: PostService



- Tiers: App server tier, Cache or In-memory tier, Storage tier
- Data Model: K-V pair
 - K: PhotoID: V: desc, img store path, other properties
- How to store in Cache tier - Hashmap
 - To store recent photos by celebs
 - K: celebID, V: a list of recent photos
- How to store in Storage tier - as a row oriented K-V store
- APIs
 - postPhoto(userID, location, time_stamp, ...)
- Algorithm in APIs
 - Generate photo id
 - Create(K.V) - CRUD
- Flow of APIs
 - Phone app to convert photo in proper resolution and format
 - Stores the img in img store, create meta-data rec in db
 - Update the celeb cache
 - Async fanout to update news feed, upload to cdn, gen reverse idx



Step 5: Recall scaling up the solution

- For each microservice
 - Check whether each tier needs to scale
 - A deterministic set of reasons can be posed across all interviews
 - Need to scale for storage (storage and cache tiers)
 - Need to scale for throughput (CPU/IO)
 - Need to scale for API parallelization
 - Need to remove hotspots
 - Availability and Geo-distribution
 - The constraints or numbers change from problem to problem
 - Solve algebraically first and then put numbers. Algebraic formula is same for all problems
 - If phase 1 takes more time, do not spend time on calculations unless asked for.

Step 5: PostService:

- For each microservice
 - A deterministic set of reasons can be posed across all interviews
 - Need to scale for storage (storage and cache tiers) - Yes
 - Need to scale for throughput (CPU/IO) - Yes
 - Need to scale for API parallelization - No
 - Need to remove hotspots - No
 - Availability and Geo-distribution - Yes
 - Solve algebraically first and then put numbers
 - Total users $\approx$ 1 billion
 - Daily active users $\approx$ 500 million
 - New photos per second $\approx$ 1000
 - Average photo file size $\approx$ 200kb
 - Let's look at each tier separately.

Step 5: PostService:

- App Tier
 - Let t = CPU work time in ms from a single thread
 - So number of operations per second handled by single thread = $1000/t$
 - Number of concurrent threads in a commodity server = 100-200 (determined by experiments)
 - So the number of API calls a server can handle per second $\sim 100000/t$, operating at full capacity.
 - But typical servers operate at 30-40% capacity, so roughly **30000/t API calls per server**.
 -
 - Assuming internet speed of 8 Mbps = 1 mega-byte per sec
 - For a 200 KB photo, we will take around 200 ms
 - Plug in $t = 200$ ms in above formula = $30000/200 = 150$ rps per server
 - For 1000 photos per sec ~ 7 servers.

Step 5: PostService:

- DB Tier
 - Image store estimation
 - Total space required for 10 years of photos (w/o replication or growth)
= 10 years * ~400 days per year * ~100k sec in a day * 1000 pic per s * 200 kb
= 80,000,000,000,000,000 b = ~80 Pb
 - (replication to be considered in step 4c)
 - Meta-data estimation
 - PhotoTable: PhotoId(int64) + UserId(int64) + title(varchar 64) + ImgStorePath(varchar 64) + ThumbnailPath (varchar 64) + PhotoLatitude(int) + PhotoLongitude(int) + TimeStamp(datetime) + numLikes(int) + numComments(int)
= 228 bytes per row * 400 billion photos ~= 100 TB w/o replication.

Step 6: Recall proposing the distributed architecture

- For each microservice
 - If a tier needs scale, scale the tier and propose the distributed system
- This is the most deterministic portion of the solution
- Steps
 - Draw a generic distributed architecture per tier
 - If app server tier and *stateless*, just round robin requests
 - If cache or storage tier
 - Partition the data into *shards* or buckets to suit requirements of scale (changes from problem to problem)
 - Propose algorithm to place shards in servers (*consistent hashing*) (same across all problems)
 - Explain how APIs work in this sharded setting (problem specific)
 - Propose replication (same across all problems)
 - Propose CP or AP (algorithms for CAP theorem do not change from problem to problem)
- This is how each microservice looks within a single data center, either on-premise or VPC in cloud

Step 6: PostService: Reasons for partitioning / sharding

1. If the size of the data is very large
2. If the query throughput is very high (and includes writes)

PhotoTable ~= 100 TB w/o replication.

Assuming 1 shard could hold 1TB, we'll need ~100 shards (w/o replication).
But what should we shard on?

- a. UserID
- b. PhotoID

Step 6: PostService:

Given a key, which node should I go to (ignore replication for now)? And once I am at the right node, how do I access the relevant record?

Option 1: Assign records to nodes randomly. Would this work?

Option 2: Based on key range

Option 3: Based on the “hash of the key” range

Rebalancing partitions

Suppose the load increases, and you add more CPUs to handle the load.

Or suppose the data size increases, and you add more RAM/Disk to store it.

Or suppose a node fails, and other machines need to take over its responsibilities.

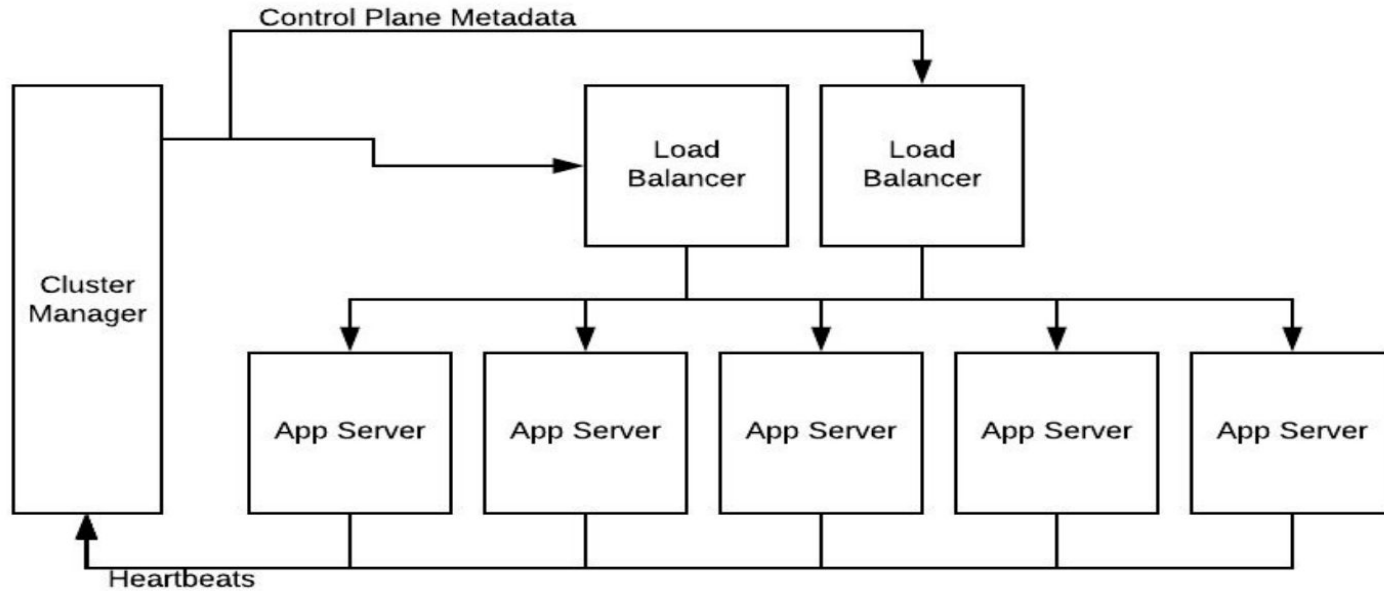
Key ---> Partition ---> Node

If Partition $\Leftrightarrow$ Node, then when a node fails, most of the keys need to be moved from node node to another.

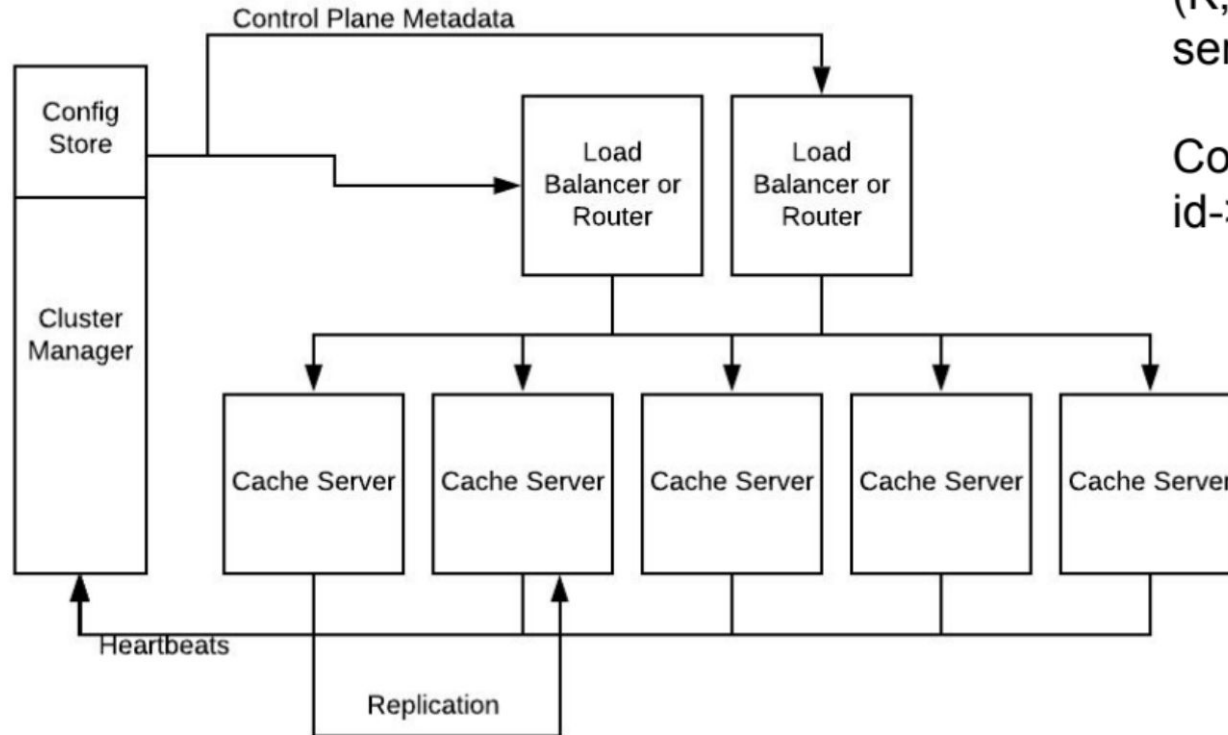
Step 6: PostService

- Sharding
 - Horizontal sharding
 - Map subsets of photos to a single shards
- Placement of shards in servers
 - Consistent hashing
- Replication
 - Yes for availability as well as for throughput
 - To reduce latency for geographically distributed users
- CP or AP?
 - AP: does not need to be strictly nanosecond level consistent
 - Best configuration: $N = 3$ or more, $R = 2$, $W = 2$, high throughput, lowest latency

Distributed Architecture - PostService



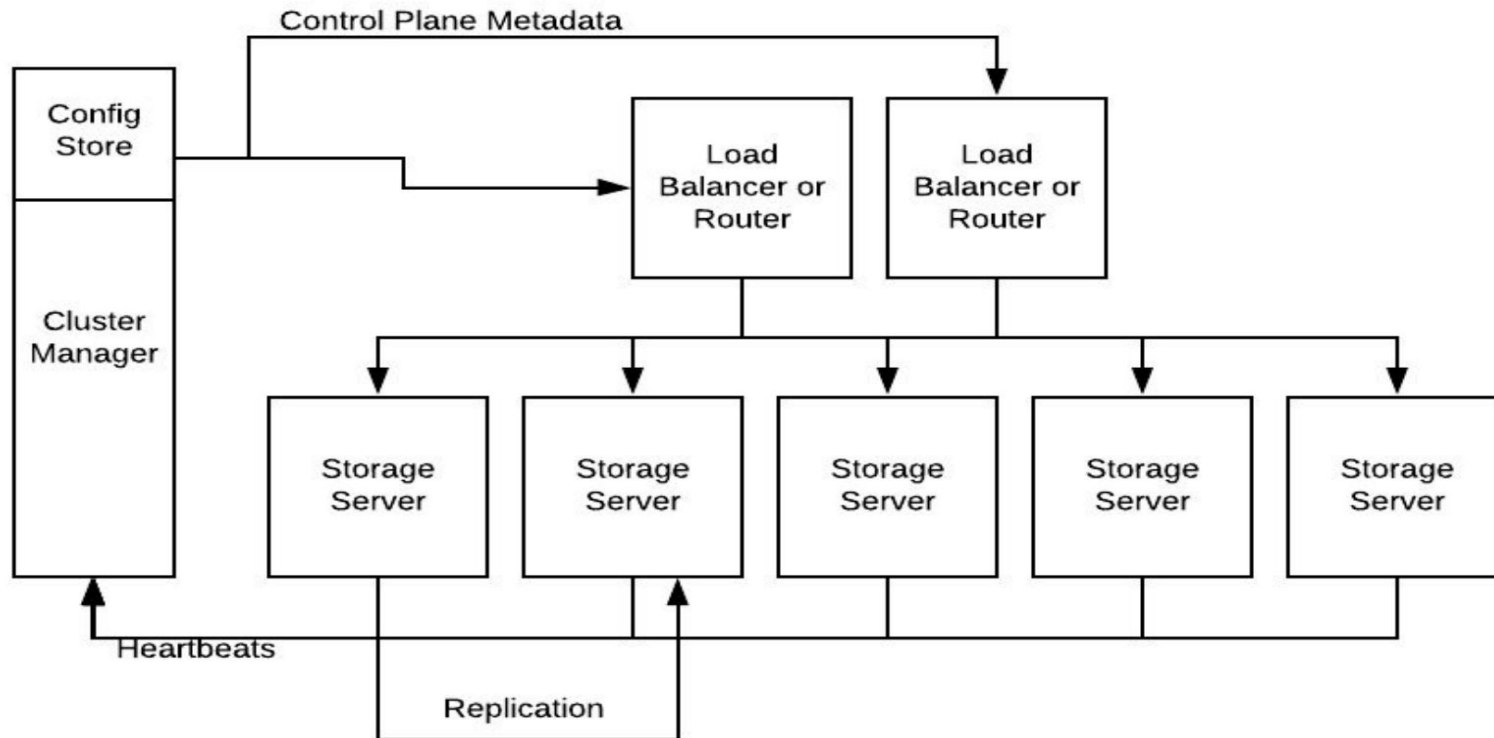
Distributed Architecture - PostService



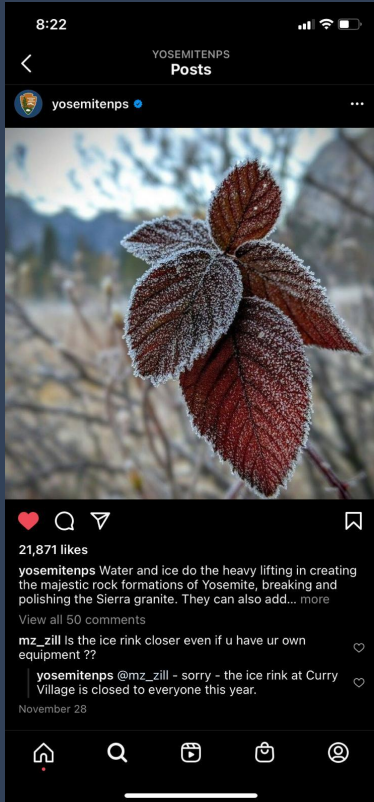
(K, V) -> partition/shard id -> servers

Config store maps shard id->servers

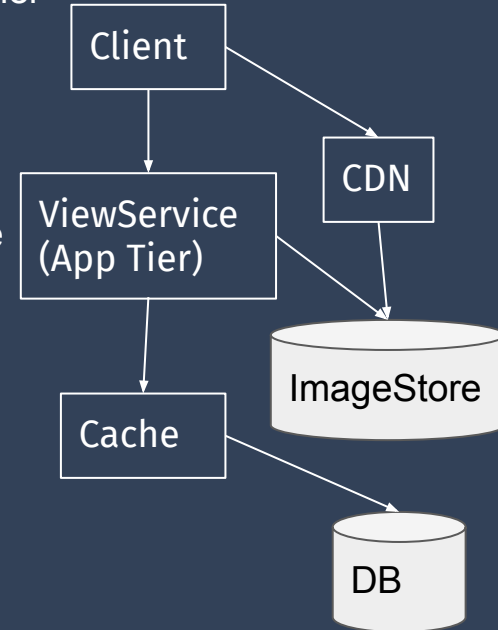
Distributed Architecture - PostService



Step 4: ViewService



- Tiers: App server tier, Cache or In-memory tier, Storage tier
- Data Model: K-V pair
 - K: PhotoID: V: desc, img store path, other properties
 - K: UserID: V: a list of photos
- How to store in Cache tier - Hashmap
- How to store in Storage tier - as a row oriented K-V store
- APIs
 - viewPhoto(PhotoID)
 - viewPhotos(UserID)
- Algorithm in APIs
 - Simple K-V workload to fetch basic data (CRUD)
 - Fetch and aggregate like status, comments
- Flow of APIs
 - Check if meta-data is available in cache
 - If yes, fetch and send it with img or cdn url
 - If not, read from db, aggregate likes, comments
 - Update cache, return to user



Step 5: ViewService: Scale up the solution

- For each microservice
 - A deterministic set of reasons can be posed across all interviews
 - Need to scale for storage (storage and cache tiers) - Yes
 - Need to scale for throughput (CPU/IO) - Yes
 - Need to scale for API parallelization - No
 - Need to remove hotspots - No
 - Availability and Geo-distribution - Yes
 - Solve algebraically first and then put numbers
 - Total users $\approx$ 1 billion
 - Daily active users $\approx$ 500 million
 - Photo views per second $\approx$ 20000
 - Average photo file size $\approx$ 200kb
 - Let's look at each tier separately.

Step 5: Need for scale in ViewService:

- App Tier
 - Let t = CPU work time in ms from a single thread
 - So number of operations per second handled by single thread = $1000/t$
 - Number of concurrent threads in a commodity server = 100-200 (determined by experiments)
 - So the number of API calls a server can handle per second $\sim 100000/t$, operating at full capacity.
 - But typical servers operate at 30-40% capacity, so roughly **30000/t API calls per server**.
 -
 - Since we are not sending the photo, but leveraging CDN, let's assume that each photo fetch takes around 50 ms.
 - Plug in $t = 50$ ms in above formula = $30000/50 = 600$ rps per server
 - For 20000 photos per sec ~ 34 servers.

Step 5: Need for scale in ViewService:

- Cache Tier
 - Even though we did DB estimation for 10 years, we don't need to store a fraction of that amount in cache.
 - We can store last 5 days worth of photos' metadata in cache
 - So $5 \text{ days} * 100,000 \text{ sec} * 1000 \text{ pic per s} \approx 500 \text{ million}$
 - $500 \text{ million} * \sim 200 \text{ bytes per pic (meta data)} \approx 100 \text{ GB}$
 - Theoretically it could fit in 2 machines with 64 GB RAM

Step 6: Distributed Architecture in ViewService

- Same as before
 - Stateless app servers
 - Shard the data
 - Consistent Hashing
 - CP or AP